



كلية الهندسة بشبرا

FACULTY OF ENGINEERING AT SHOUBRA

FACEBOOK PROJECT REPORT

Members

- AbdelrahmanWahid
Abdallah
- MohamedWahid
Abdallah
- Aya Mohamed
- Abdelrahman
Mamdouh Mohamed
- Yassin Ahmed Nasr

Dr. Walaa Gouda

Table of Contents

[1. Introduction](#)

[2. Technologies Used](#)

[3. Posts Section](#)

3.1 Create Post

- **3.2 Edit Post**
- **3.3 Delete Post**
- **3.4 Share Post**
- **3.5 Preview Media in Modal**

[4. Reacts Section](#)

- **4.1 Like Post**
- **4.2 Dislike Post**
- **4.3 Fetch Current Reacts**

1. Introduction

The FaceTook Project is a social media application designed to simulate the core features of a platform like Facebook, enabling users to share content, interact with posts, and engage with others in a community. This project has been developed using a combination of PHP, Laravel, Vue.js, and MySQL, offering both backend and frontend functionalities essential for creating a seamless user experience.

- The project includes features like:

- Creating, editing, and deleting posts
- Uploading multiple types of media (images, videos) along with text
- Sharing posts (posts can contain other posts)
- Reacting to posts (likes)
- Viewing and interacting with post media in modals
- editing current profile (update name , profile , cover pictures , bio)
- adding comments to current posts
- adding replies to comments
- follow other users and see their posts and showing current followers

2. Technologies Used

Backend: PHP (Laravel)

Frontend: Vue.js and Laravel Blade

Database: MySQL

Development Environment: XAMPP (Apache, MySQL, PHP)

3. Posts Section

The **Posts** section is one of the central features of the social media platform, enabling users to create, manage, and interact with posts. In this section, users can:

1. **Create Posts:** Allowing users to share their thoughts, ideas, images, and videos with their friends and followers.
2. **Upload Media:** Users can upload multiple media files (images, videos) along with the post. Posts can contain a combination of text and media or media alone.
3. **Edit Posts:** Users can modify the content of their existing posts.
4. **Delete Posts:** Users can remove their posts from the platform.
5. **Share Posts:** Users can share posts from other users, allowing the post to appear on their own profile, making the original post visible to other audience.
6. **Preview Posts Media in Modal:** Users can view the media (image or video) attached to a post before submitting it, via a modal popup.

1. Creating a Post and uploading

Media:

Creating a post is the core feature that allows users to express themselves.

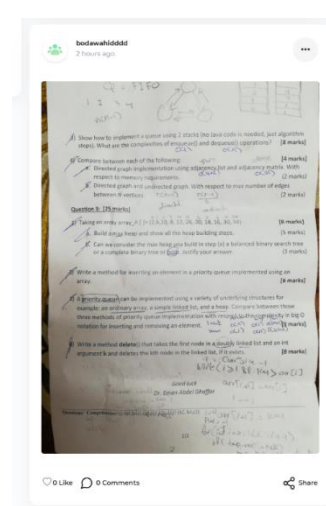
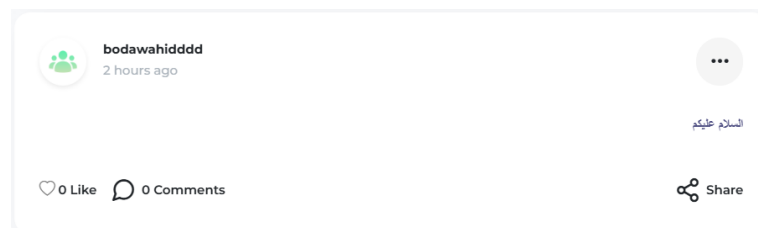
The post can be:

- **Text only**
- **Media only.**
- **Text and Media.**

- The system supports different types of media:

- **Images:** JPEG, PNG, GIF files.

- **Videos:** MP4, AVI, or other popular video formats



Users can upload one or more media files with their posts. The platform handles the uploading process, which typically includes:

- **Multiple file selection:** Users can select multiple files simultaneously from their file system.
- **File validation:** The system ensures that only acceptable media types are uploaded and checks for file size limits.
- **Storing media files:** Once uploaded, the files are stored on the server and linked to the post in the database.

- The code of creation and storing media process :

```
public function create(Request $request)
{
    $request->validate([
        'content' => ['nullable', 'string'],
        'content_type' => ['nullable', 'string', Rule::in(['text', 'attachement'])],
        'media' => 'nullable|array',
        'media.*' => ['nullable', 'file', 'max:10000'],
    ]);
    $path = null;
    DB::insert('insert into posts (content, content_type, user_id, created_at) values (?, ?, ?, ?)', [$request->content, $request->content_type ?? 'text', Auth::id(), now()]);
    $postId = DB::getPdo()->lastInsertId();
    foreach ($request->media as $file) {
        $originalName = $file->getClientOriginalName();
        $uniqueFilename = time() . '_' . $originalName;
        $file->storeAs('posts/media', $uniqueFilename, ['disk' => 'public']);
        $content_type = $file->getClientMimeType();
        if (str_contains($content_type, 'image')) {
            $content_type = 'image';
        } else {
            $content_type = 'video';
        }
        DB::insert('insert into posts_media (post_id, media_path, media_type, uploaded_at) values (?, ?, ?, ?)', [$postId, $uniqueFilename, $content_type, now()]);
    }
    // selecting data
    return redirect()->route('get.data');
```

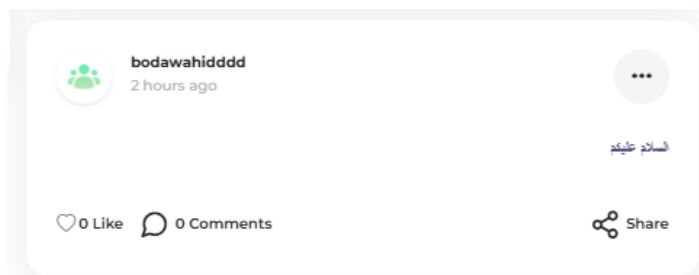
- The method handles the creation of a new post, including optional text and media (images or videos).
- It validates the incoming request data to ensure that it meets the specified requirements.
- The post data is inserted into the posts table, and the media files (if any) are uploaded to the server and stored in the posts_media table, linking the media with the corresponding post.
- The user is redirected to a specified route (get.data) after successfully creating the post and uploading the media.

2. Editing a Post

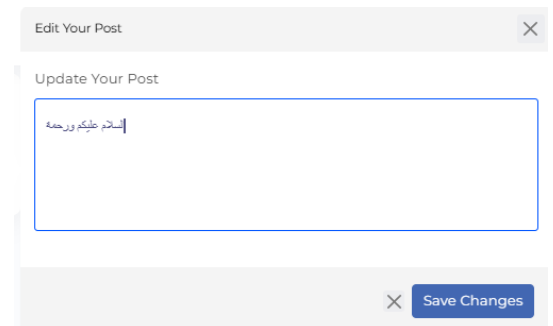
Editing a post gives users the ability to modify or update the content of a post after it has been created. The editing process allows users to:

Editing provides flexibility for users who want to correct or improve the content of their posts after they have been shared.

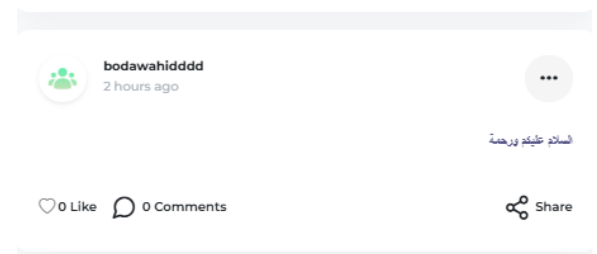
- Original post



editing



after edit



- The code of editing posts

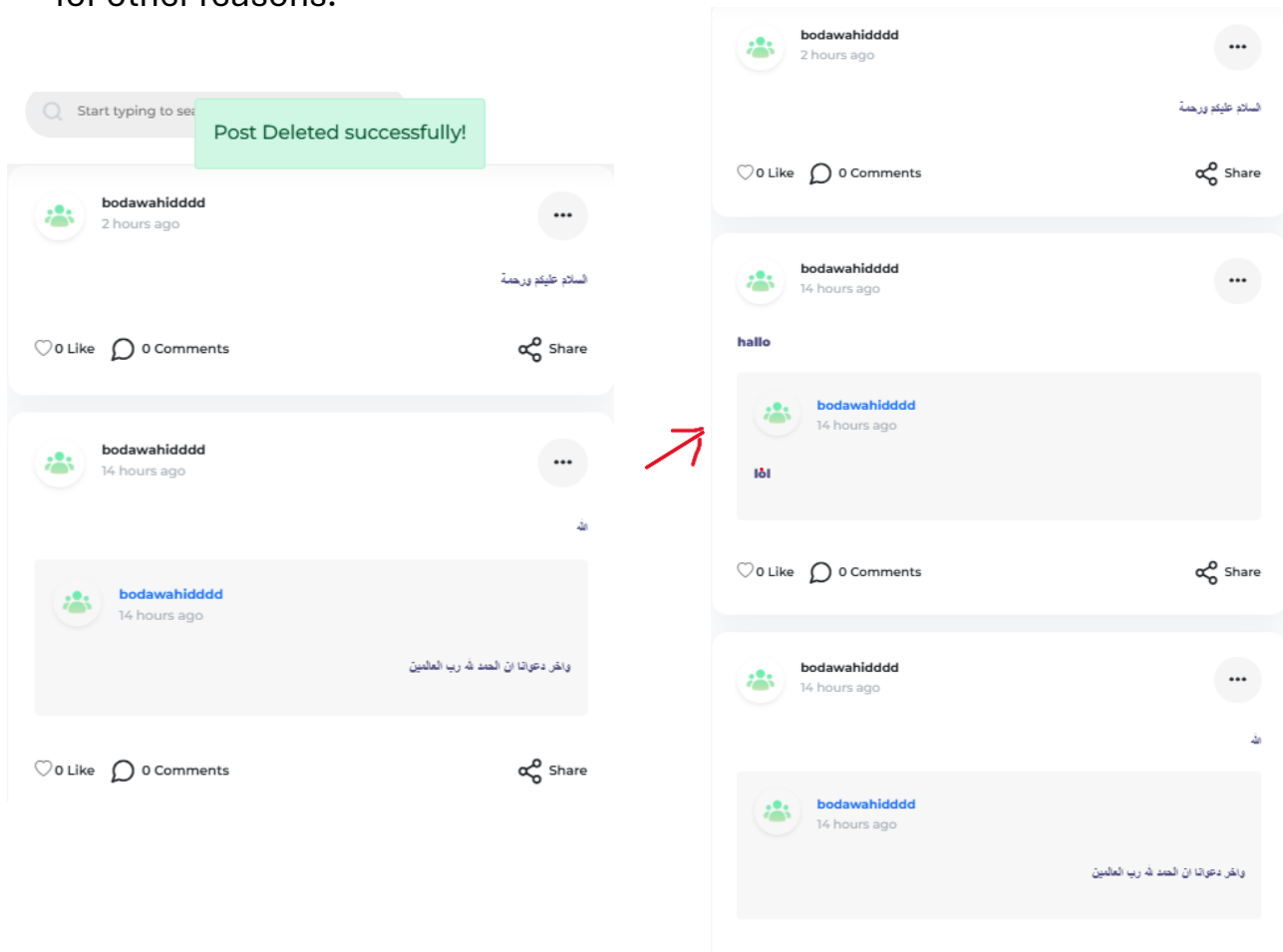
- This function is designed to edit an existing post by updating its content field in the database.
- The post ID and new content are validated to ensure that the provided post ID exists and that the content is a valid string.
- The post's content is updated in the posts table, and the updated_at timestamp is modified to reflect the time of the update.
- A success message is returned to indicate that the post was updated successfully.

```
public function editPost(Request $request)
{
    $request->validate([
        'post_id' => ['required', 'integer', 'exists:posts,id'],
        'content' => ['required', 'string'],
    ]);
    DB::update('update posts set content = ? , updated_at = ? where id = ?', [$request->content, now(), $request->post_id]);
    // $post = DB::select('select * from posts where id = ?', [$request->post_id]);
    return response()->json(['message' => 'post is updated successfully']);
}
```

3. Deleting a Post

The option to delete a post allows users to remove their content from the platform.

This feature is critical for user control over their shared content, enabling them to delete posts that are no longer relevant or that they wish to remove for other reasons.



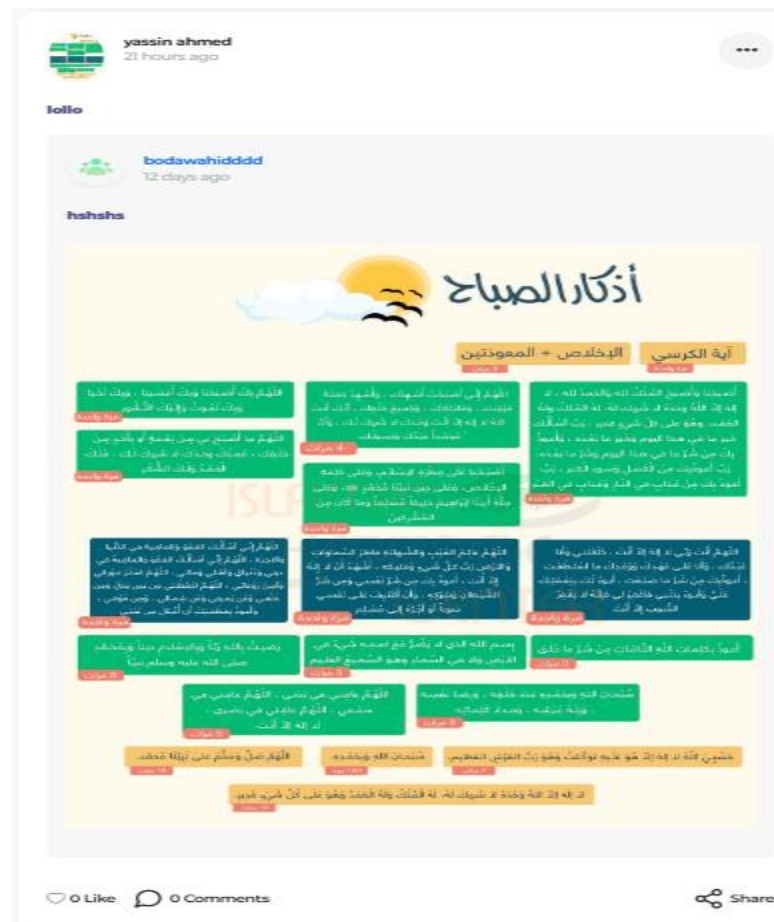
- The code of deleting posts

```
public function permanentDelete(Request $request)
{
    $request->validate([
        'post_id' => ['required', 'integer', 'exists:posts,id'],
    ]);
    DB::delete('delete from posts where id = ?', [$request->post_id]);
    return response()->json(['message' => 'post is deleted successfully']);
}
```

- The function handles the permanent deletion of a post from the database.
- It validates the post_id to ensure it exists in the database.
- It then deletes the post with the specified post_id from the posts table.
- A success message is returned in the response to confirm the post was deleted.

4. Sharing a Post

- The ability to share posts is a common social media feature that allows users to share content from other users with their own followers. The shared post will appear on the user's profile or feed, and it typically includes a reference to the original author.
- **The code of sharing posts**
 - The function allows a user to **share a post** by creating a new post that references the original post via post_id.
 - It validates the request to ensure that a valid post ID is provided and that the content (if provided) is a string.
 - The new post is inserted into the posts table with a reference to the original post (post_id) and the authenticated user's ID.

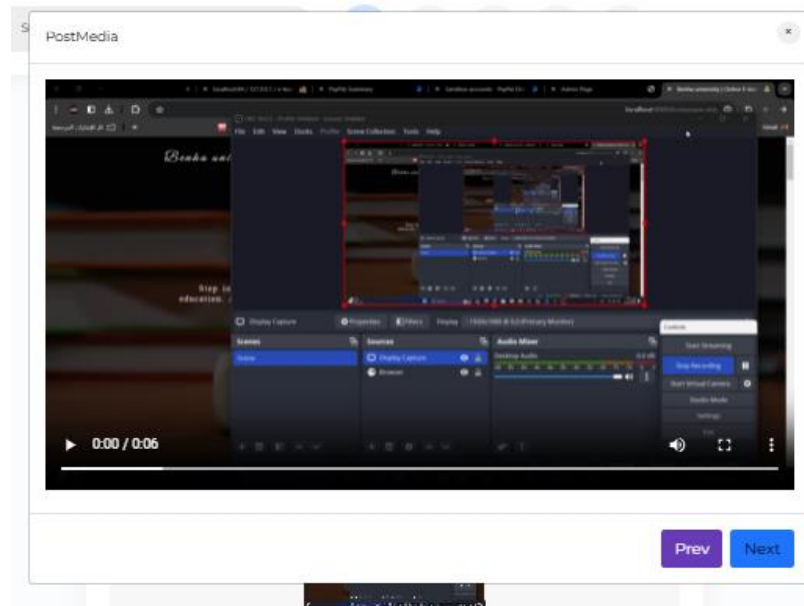


```
public function sharePost(Request $request)
{
    $request->validate([
        'content' => ['nullable', 'string'],
        'post_id' => ['required', 'numeric', 'exists:posts,id'],
    ]);
    DB::insert('insert into posts (content,content_type, user_id , created_at,post_id) values(?,?,?,?,?)', [$request->content ?? null, 'text', Auth::id(), now(), $request->post_id]);
    return redirect()->route('get.data');
}
```


- After sharing the post, the user is redirected to a route (`get . data`) that displays the shared content and other data.

5. Previewing Media in a Modal

A **media preview** feature allows users to view images or videos of already existing post. The modal popup is a user interface element that displays media content in a separate window, making it easier for users to preview their uploads.



Posts Table From MYSQL

And its relation with user and itself (sharing posts)

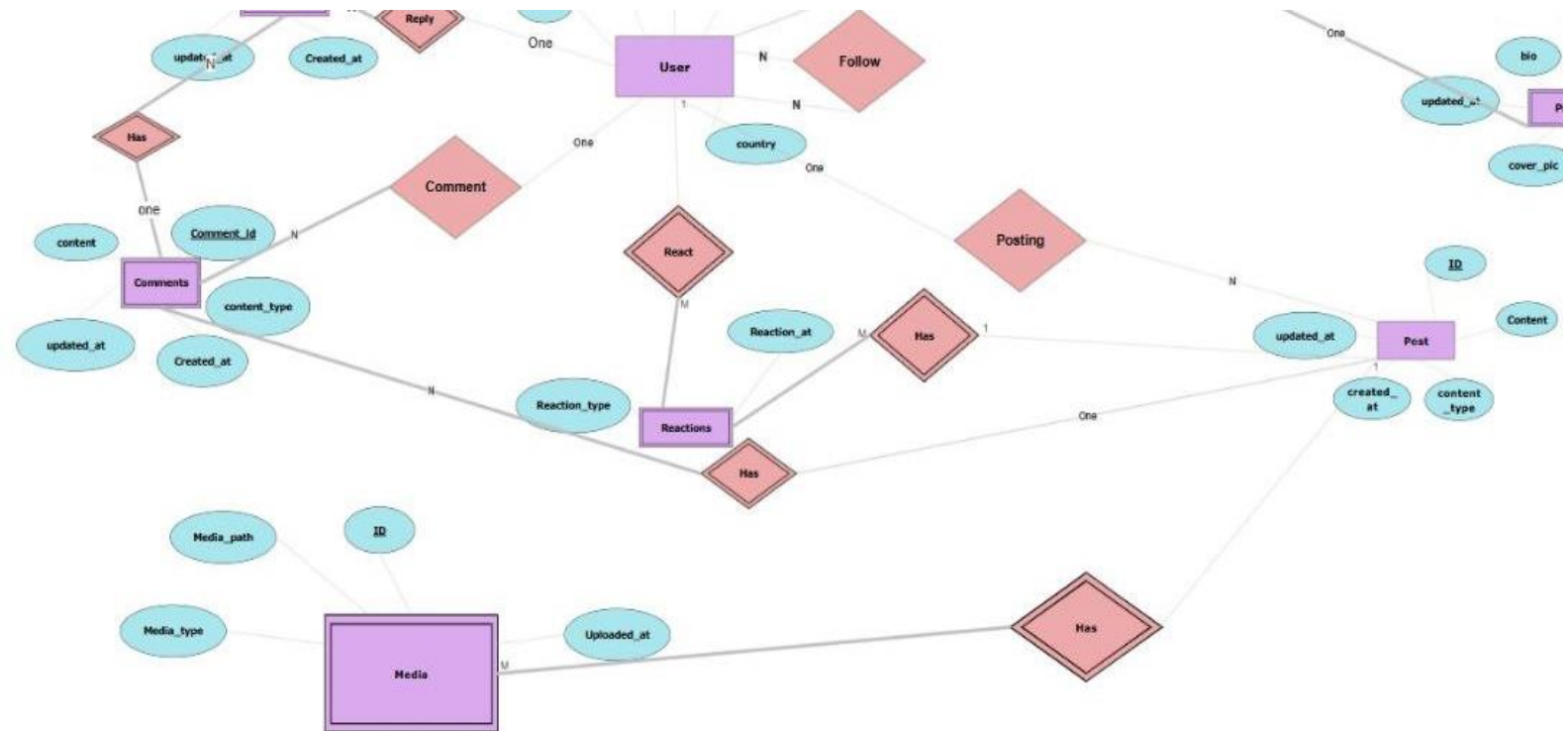
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id 🔑	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	content	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
☐ 3	content_type	enum('text', 'attachement')	utf8mb4_unicode_ci		No	text			Change Drop More
☐ 4	user_id 🔑	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More
☐ 5	created_at	timestamp			Yes	NULL			Change Drop More
☐ 6	updated_at	timestamp			Yes	NULL			Change Drop More
☐ 7	deleted_at	timestamp			Yes	NULL			Change Drop More
☐ 8	post_id 🔑	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More

PostsMedia Table From MYSQL

And its relation with posts table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id 🔑	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	post_id 🔑	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 3	media_path	text	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 4	media_type	enum('image', 'video')	utf8mb4_unicode_ci		No	None			Change Drop More
☐ 5	uploaded_at	timestamp			No	current_timestamp()			Change Drop More

ER Diagram of the posts entity with other entities



4. Reacts Section

Reacts Section: Likes

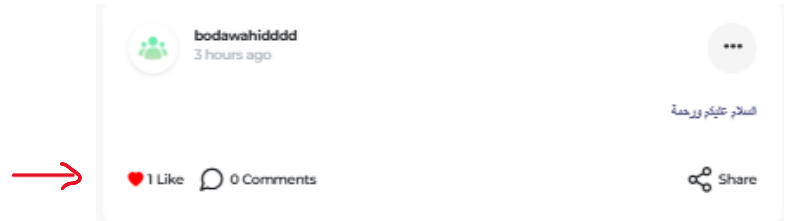
The **React** section allows users to interact with posts by **liking** them. This functionality includes adding, removing, and updating the number of likes a post receives, as well as keeping track of which users have liked a post.

1. Adding a Like to a Post

When a user clicks the "like" button on a post, the following actions are triggered:

- **Like Addition:** If the user hasn't already liked the post, the system adds a new like for that post in the database.

- **Tracking Liked Users:** The system records which user has liked the post by associating the user's ID with the post's ID. This allows the system to know exactly which users have liked each post.
- **Incrementing Like Count:** The number of likes on the post is increased by one.



The code of Adding react

```
public function addPostLikes(Request $request)
{
    $request->validate([
        'post_id' => ['required', 'integer', 'exists:posts,id'],
    ]);
    DB::insert('insert into reacts (post_id, user_id) values (?, ?)', [$request->post_id, Auth::id()]);
    return response()->json(['message' => 'reacted successfully']);
}
```

-This function allows a user to like a post by storing the post's ID and the user's ID in the reacts table. If the operation is successful, a confirmation message is returned.

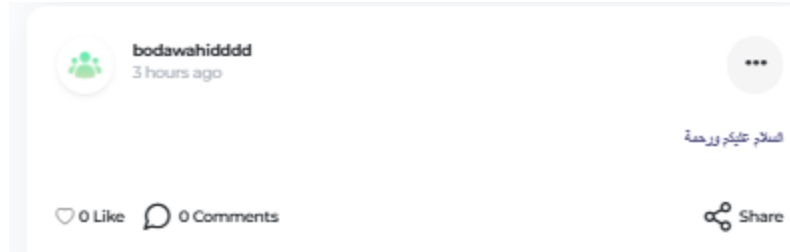
2. Removing a Like from a Post

When a user clicks the "like" button again (to remove the like), the following actions occur:

- **Like Removal:** The system removes the like associated with the user and the post. This is typically done by deleting the record from the **likes** table that associates the user with the post.

- **Decrementing Like Count:**

After the like is removed, the number of likes on the post decreases by one. This updated count is also reflected in the database and on the user interface.



- The code of removing reacts:

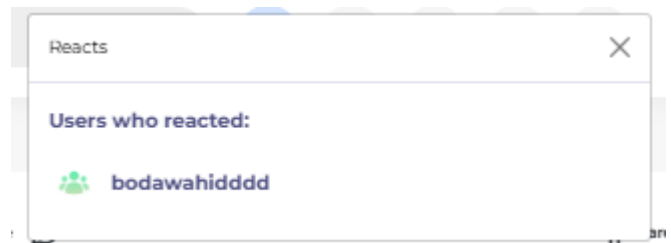
```
public function removePostLikes(Request $request)
{
    $request->validate([
        'post_id' => ['required', 'integer', 'exists:posts,id'],
    ]);
    DB::delete('delete from reacts where post_id = ? and user_id = ?', [$request->post_id, Auth::id()]);
    return response()->json(['message' => 'deleted react successfully']);
}
```

This function allows a user to **unlike a post** by deleting their like record from the reacts table based on the post's ID and their user ID. If the operation is successful, a confirmation message is returned.

3. Fetching and Displaying Liked Users

In addition to showing the number of likes, it's often necessary to display the users who have liked the post. This can be done by:

- **Fetching Liked Users:** When a user clicks on the "like" button, the system can fetch the list of users who have liked the post from the database. This list can then be displayed either as a list of usernames or user avatars.
- **Displaying Liked Users in the UI:** A modal or dropdown can be used to show the names or avatars of the users who have liked the post. This allows users to see who interacted with the content, encouraging more engagement.



- The code of fetching and displaying liked users:

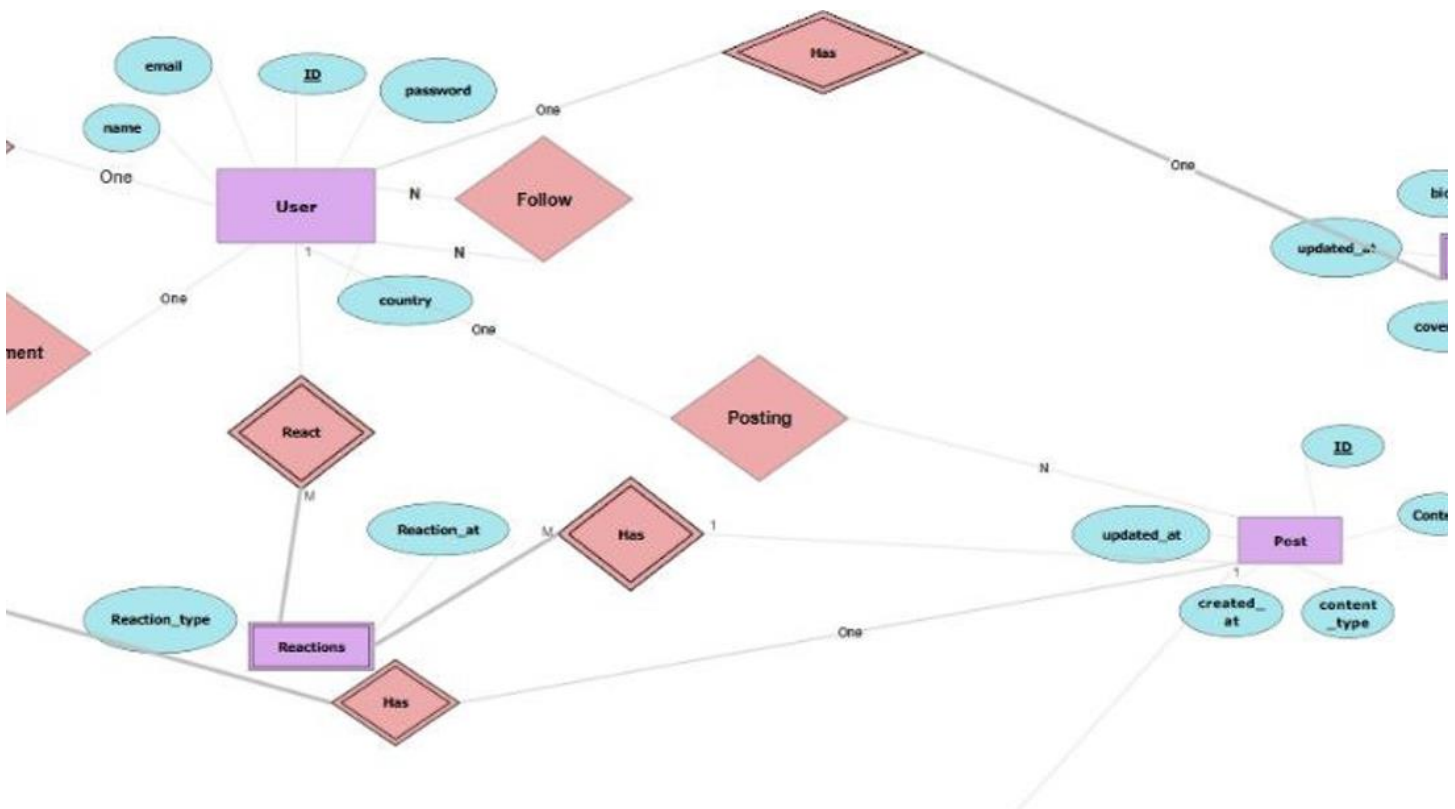
```
public function getReactsInfo(Request $request)
{
    $request->validate([
        'post_id' => ['required', 'integer', 'exists:posts,id'],
    ]);
    $reactedUsers = DB::select('select u.id , u.name , p.profile_picture
    from reacts r join users u on r.user_id = u.id and r.post_id = ? left join profiles p on u.id = p.user_id ', [$request->post_id]);
    return response()->json($reactedUsers);
}
```

- This function retrieves and returns a list of users who have liked a specific

Reacts Table From MYSQL

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
☐ 2	post_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 3	user_id	bigint(20)		UNSIGNED	No	None			Change Drop More
☐ 4	react_type	enum('like', 'haha', 'love', 'wow', 'sad')	utf8mb4_unicode_ci		No	like			Change Drop More
☐ 5	created_at	timestamp			Yes	NULL			Change Drop More
☐ 6	updated_at	timestamp			Yes	NULL			Change Drop More

- ER Diagram of Reacts Entity with other entities

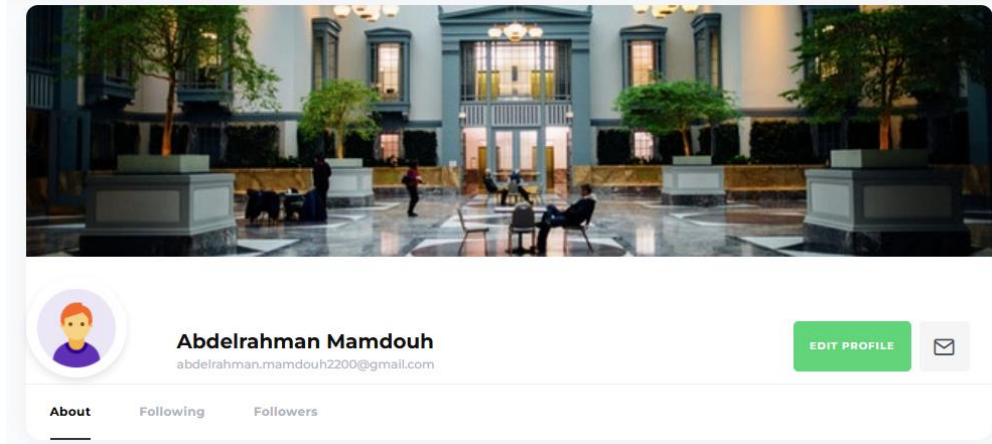


The profileInformation.vue is used for rendering a user profile page. It has several features:

- Display user information
- Following/Followers functionality
- Edit profile
- View people the user following and the ones who follow him

User Profile Display:

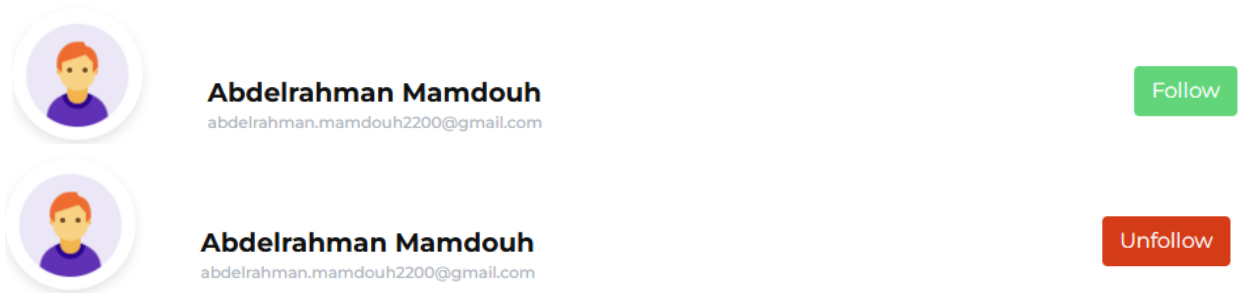
- Displays the user's cover picture and profile picture. If there's no image it gives the user a default one
- Shows the user's name and email address.



- This button only appears to the logged user

Follow/Unfollow Button:

- Visible when viewing another user's profile.
- Toggles between "Follow" and "Unfollow" states dynamically based on the `$parent.isFollowing` property.



Edit Profile Functionality:

- Available for the logged-in user when viewing their own profile. It opens a pop up and the name field will be filled with the user name as a default value
- Opens a modal to edit:
 - Name.
 - Profile image.
 - Cover image.
- Includes live image preview in the edit model.

Edit Profile

Name:

ahmed mohamed

Profile Image:

Choose File

No file chosen

Cover Image:

Choose File

No file chosen

Save Changes

Cancel

Followers and Following Modals:

- Buttons to view a list of followers and the users being followed.
- Each modal lists users with their profile picture and name. it opens in a pop up over the page
- Handles empty states (e.g., "No followers yet" or "Not following anyone yet").

Followers



Abdelrahman Mamdouh

Following



Not following anyone yet

Joined At section:

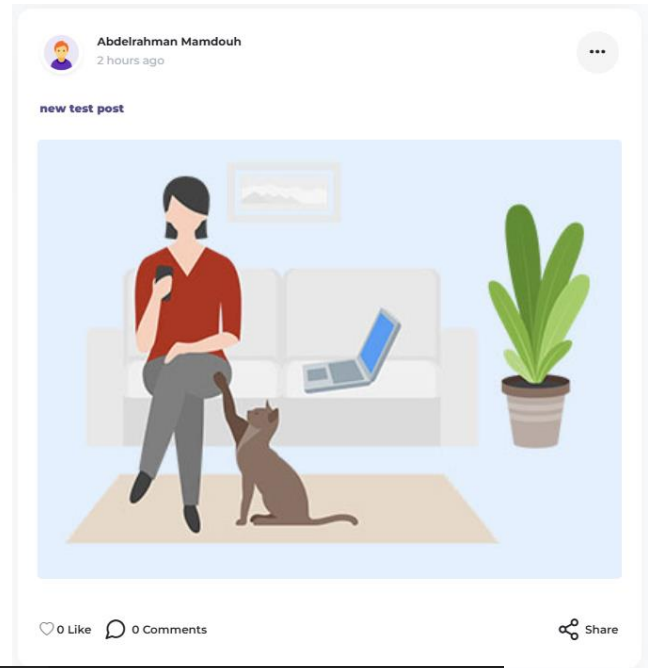
- It views when the user joined the app



Joined At 2024-12-17

View created or shared posts on the account

- We can also view the posts the user created or the one he shared



The update profile function

```
public function updateProfile(Request $request): JsonResponse|mixed
{
    // return $request->cover_image;
    $request->validate(rules: [
        'name' => ['string', 'max:255'],
        'profile_image' => ['nullable', 'file', 'mimes:jpg,jpeg,png', 'max:2048'],
        'cover_image' => ['nullable', 'file', 'mimes:jpg,jpeg,png', 'max:4096'],
    ));

    $userId = Auth::id();

    // Initialize variables for file paths
    // $profileImagePath = null;
    // $coverImagePath = null;
    $profile_image_name = null;
    $cover_image_name = null;
    // Handle Profile Image Upload
    if ($request->hasFile(key: 'profile_image')) {
        $file = $request->file(key: 'profile_image');
        $originalName = $file->getClientOriginalName();
        // $extension = $file->getClientOriginalExtension();
        $profile_image_name = time() . '_' . $originalName;
        $file->storeAs(path: 'users/image', name: $profile_image_name, options: ['disk' => 'public']);
    }

    // Handle Cover Image Upload
}
```

Input validation

- **name:** Must be a string and have a maximum length of 255 characters.
- **profile_image:** Optional (nullable), must be a file of type jpg, jpeg, or png, with a size limit of 2 MB.
- **cover_image:** Optional, must be a file of type jpg, jpeg, or png, with a size limit of 4 MB.


```
// Handle Cover Image Upload
if ($request->hasFile(key: 'cover_image')) {
    $file = $request->file(key: 'cover_image');
    $originalName = $file->getClientOriginalName();
    // $extension = $file->getClientOriginalExtension();
    $cover_image_name = time() . '_' . $originalName;
    $file->storeAs(path: 'users/image', name: $cover_image_name, options: ['disk' => 'public']);
}

// Update the name in the users table
if ($request->name != null) {
    DB::update(
        query: 'update users set name = ?, updated_at = ? where id = ?',
        bindings: [$request->name, now(), $userId]
    );
}

if ($profile_image_name && $cover_image_name) {
    DB::update(query: 'update profiles set profile_picture = ? , cover_picture = ? where user_id = ? ', bindings: [$profile_image_name, $cover_image_name, Auth::id()]);
} else if ($profile_image_name) {
    DB::update(query: 'update profiles set profile_picture = ? where user_id = ? ', bindings: [$profile_image_name, Auth::id()]);
} else if ($cover_image_name) {
    DB::update(query: 'update profiles set cover_picture = ? where user_id = ? ', bindings: [$cover_image_name, Auth::id()]);
}

// Update the profile_picture and cover_picture in the profiles table

return response()->json(data: ['message' => 'Profile updated successfully']);
}
```

Handling Profile picture

- Checks if a profile image file is uploaded.
- If that's the case, it Gets the original filename of the uploaded file.
- Create a unique filename by appending a timestamp to avoid conflicts.
- Stores the file in the users/image directory within the public disk.
- The same goes for the cover image

Update the username

- Checks if a new name is provided.
- Update the name column and updated_at timestamp in the users table for the authenticated user.

Updating Profile Images in Database

- If both profile and cover images are uploaded, updates both.
- If only one of them is uploaded, updates only the given column.

Finally

- Returns a JSON response with a success message.

```
return response()->json(data: ['message' => 'Profile updated successfully']);
```

Follow Function

```
public function follow(Request $request): JsonResponse|mixed
{
    $request->validate(rules: [
        'user_id' => ['required', 'integer', 'exists:users,id'],
    ]);

    $followerID = Auth::id();

    $exists = DB::select(
        query: 'select 1 from following where follower_id = ? and following_id = ? limit 1',
        bindings: [$followerID, $request->user_id]
    );

    if (!empty($exists)) {
        return response()->json(data: ['message' => 'Already following'], status: 200);
    }

    DB::insert(
        query: 'insert into following (following_id, follower_id ,followed_at) values (?, ?, ?)',
        bindings: [$request->user_id, $followerID, now()]
    );

    return response()->json(data: ['message' => 'followed successfully']);
}
```

Yassin Ahmad Nasr, 2 days ago • follow

It allows a user to follow another user

It ensures:

- The user being followed exists.
- The authenticated user is not already following the target user.
- A new "follow" relationship is stored in the database if valid.

Validation

Validates the incoming request to ensure:

- user_id is provided (required).
- user_id is an integer (integer).
- user_id exists in the users table (exists:users,id).

Check Existing Follow Relationship

- SQL query to check if the authenticated user is already following the target user:
 - Looks for a record in the following table where follower_id matches the current user and following_id matches the target user.
 - Uses select 1 for efficiency since the actual data isn't needed, only the existence of the record.
 - Adds limit 1 to optimize the query by stopping at the first match.

If a match is found, the \$exists array will not be empty.

If the user already followed, the function immediately returns a JSON response with the message "Already following" and a 200 status code.

Insert New Follow Relationship

Inserts a new record into the following table:

- following_id: The ID of the user being followed.
- follower_id: The ID of the authenticated user.
- followed_at: The current timestamp, representing when the follow action occurred.

The Database Tables

Table	Action	Rows	Type	Collation	Size	Overhead
☐ cache	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ cache_locks	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ comments	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ failed_jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ following	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ job_batches	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ migrations	★ Browse Structure Search Insert Empty Drop	13	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ password_reset_tokens	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ personal_access_tokens	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ posts	★ Browse Structure Search Insert Empty Drop	45	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ posts_media	★ Browse Structure Search Insert Empty Drop	20	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ profiles	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ reacts	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ replies	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ sessions	★ Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ users	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
17 tables	Sum	97	InnoDB	utf8mb4_general_ci	576.0 KiB	0 B

Comments and Replies Section

The Comments and Replies section is a key feature that allows users to engage with posts, express their opinions, and reply to others. In this section, users can:

1. Create Comments:

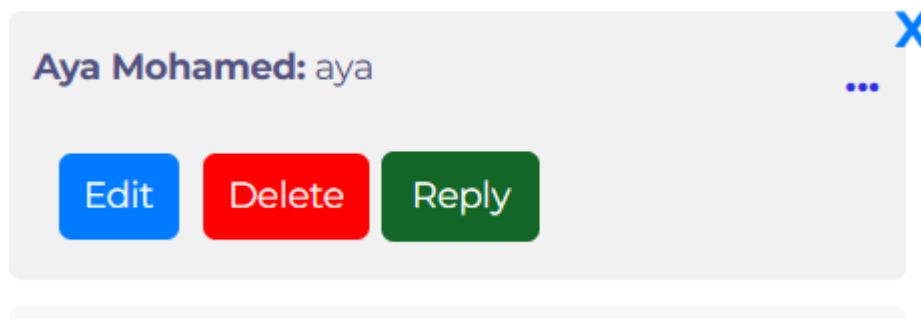
Allowing users to add comments on posts to share their thoughts, ask questions, or start discussions.

2. Edit Comments:

Users can edit their own comments to correct errors, update their opinion, or modify their input.

3. Delete Comments:

Users can delete their own comments from the platform if they no longer wish to participate in the conversation or if they made a mistake.



4. Reply to Comments:

Users can reply to others' comments, allowing them to continue the conversation or provide additional context, forming a thread of replies.

5. Edit Replies:

Users can modify their replies to comments if they want to clarify their response, correct mistakes, or expand on their thoughts.

6. Delete Replies:

Users can delete their replies to comments, removing their contribution from the thread if they choose to retract it or if they no longer wish to participate.

Write a Comment

Write a comment...

Post Comment

Create Comments:

Allowing users to add comments on posts to

share their thoughts, ask questions, or start discussions

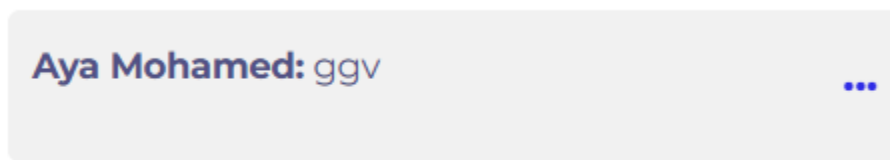
```
public function addComment(Request $request)
{
    // dump($request->all());
    $validated = $request->validate([
        'post_id' => ['required', 'numeric', 'exists:posts,id'],
        'content' => ['required', 'string'],
    ]);

    $commentId = DB::table('comments')->insertGetId([
        'post_id' => $validated['post_id'],
        'user_id' => Auth::id(),
        'content' => $validated['content'],
        'created_at' => now(),
        'updated_at' => now(),
    ]);

    return response()->json(['id' => $commentId, 'message' => 'Comment added successfully']);
}
```

The code of adding comments

The addComment function handles the process of adding a new comment to a post. It first validates the incoming data (ensuring the post_id exists and content is provided). Then, it inserts the comment into the comments table with the authenticated user's ID, the post ID, and the content. Finally, it returns a JSON response containing the ID of the newly added comment and a success message. This function enables users to comment on posts on the platform



- **Edit Comments:**

Users can edit their own comments to correct errors, update their opinion, or modify their input

```
public function updateComment(Request $request, $commentId): JsonResponse|mixed
{
    $validated = $request->validate(rules: [
        'content' => 'required|string',
    ]);

    $comment = DB::table(table: 'comments')->where(column: 'id', operator: $commentId)->where(column: 'user_id', operator: $user_id)->first();

    if (!$comment) {
        return response()->json(data: ['message' => 'Comment not found or you are not authorized to edit this comment'], status: 404);
    }

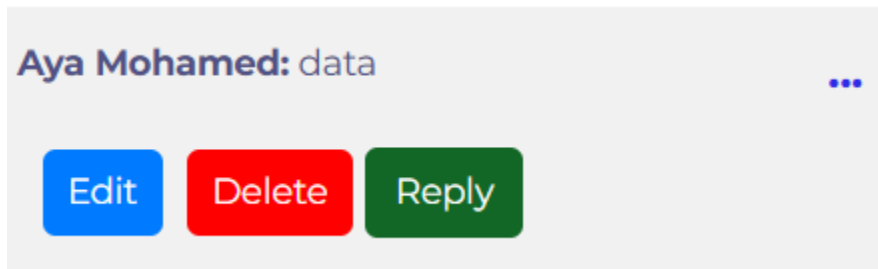
    DB::table(table: 'comments')
        ->where(column: 'id', operator: $commentId)
        ->update(values: [
            'content' => $validated['content'],
            'updated_at' => now(),
        ]);

    return response()->json(data: ['message' => 'Comment updated successfully'], status: 200);
}
```

The code of Edit comments

The updateComment function allows a user to edit their own comment. It validates the new content, checks if the comment exists and belongs to the authenticated user, and then updates the comment's content in the database.

If successful, it returns a success message; otherwise, it returns an error message.



- **Delete Comments:**

Users can delete their own comments from the platform if they no longer wish to participate in the conversation or if they made a mistake.

```
public function deleteComment($commentId): mixed
{
    $comment = DB::table('comments')->where('id', $commentId)->where('user_id', Auth::id());

    if (!$comment) {
        return response()->json(['message' => 'Comment not found or you are not authorized to delete this comment']);
    }

    DB::table('comments')->where('id', $commentId)->delete();

    DB::table('replies')->where('comment_id', $commentId)->delete();

    return response()->json(['message' => 'Comment deleted successfully']);
}
```

The code of Delete comments

The deleteComment function allows a user to delete their own comment. It first checks if the comment exists and belongs to the authenticated user. If valid, it deletes the comment and any replies associated with it from the replies table. Finally, it returns a success message.

Aya Mohamed: data ...

Edit Delete Reply

Are you sure you want to delete this comment?

Yes, Delete Cancel

Write a Comment

Aya Mohamed: aya ... X

Write a Comment

Write a comment...

Reply to Comments:

Users can reply to others' comments, allowing them to continue the conversation or provide additional context, forming a thread of replies.

Post Comment

Aya Mohamed: gg v ...

Aya Mohamed: ahmeed ...

Edit Delete

Aya Mohamed: data ...

Edit Delete Reply


```

public function addReply(Request $request, $commentId): JsonResponse|mixed {
    $validated = $request->validate(rules: [
        'content' => 'required|string',
    ]);

    $comment = DB::table(table: 'comments')->where(column: 'id', operator: $commentId)->first();
    if (!$comment) {
        return response()->json(data: ['message' => 'Comment not found'], status: 404);
    }

    $replyId = DB::table(table: 'replies')->insertGetId(values: [
        'comment_id' => $commentId,
        'user_id' => Auth::id(),
        'content' => $validated['content'],
        'created_at' => now(),
        'updated_at' => now(),
    ]);

    return response()->json(data: ['id' => $replyId, 'message' => 'Reply added successfully']);
}

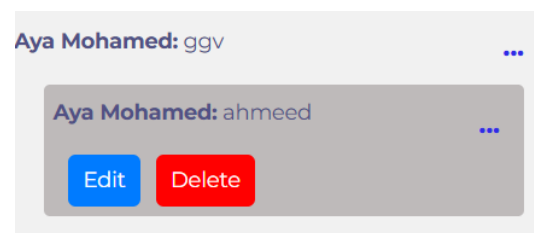
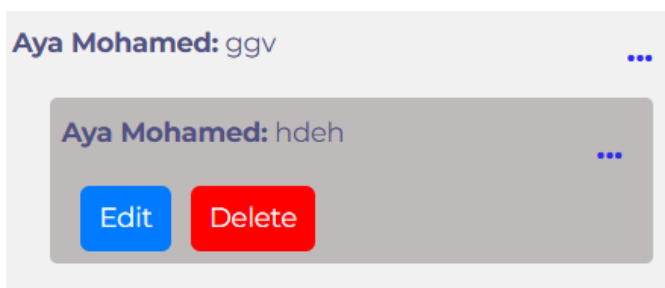
```

The code of adding Reply

The addReply function allows a user to add a reply to an existing comment. It validates the reply content, checks if the comment exists, and then inserts the reply into the replies table. If successful, it returns the ID of the new reply and a success message.

- **Edit Replies:**

Users can modify their replies to comments if they want to clarify their response, correct mistakes, or expand on their thoughts.



```

public function updateReply(Request $request, $commentId, $replyId): JsonResponse|mixed {
    $validated = $request->validate(rules: [
        'content' => 'required|string',
    ]);

    $reply = DB::table('replies')->where('id', operator: $replyId)->where('comment_id', operator: $commentId)->first();
    if (!$reply) {
        return response()->json(data: ['message' => 'Reply not found or you are not authorized to edit this reply'], status: 404);
    }

    DB::table('replies')
        ->where('id', operator: $replyId)
        ->update(values: [
            'content' => $validated['content'],
            'updated_at' => now(),
        ]);

    return response()->json(data: ['message' => 'Reply updated successfully']);
}

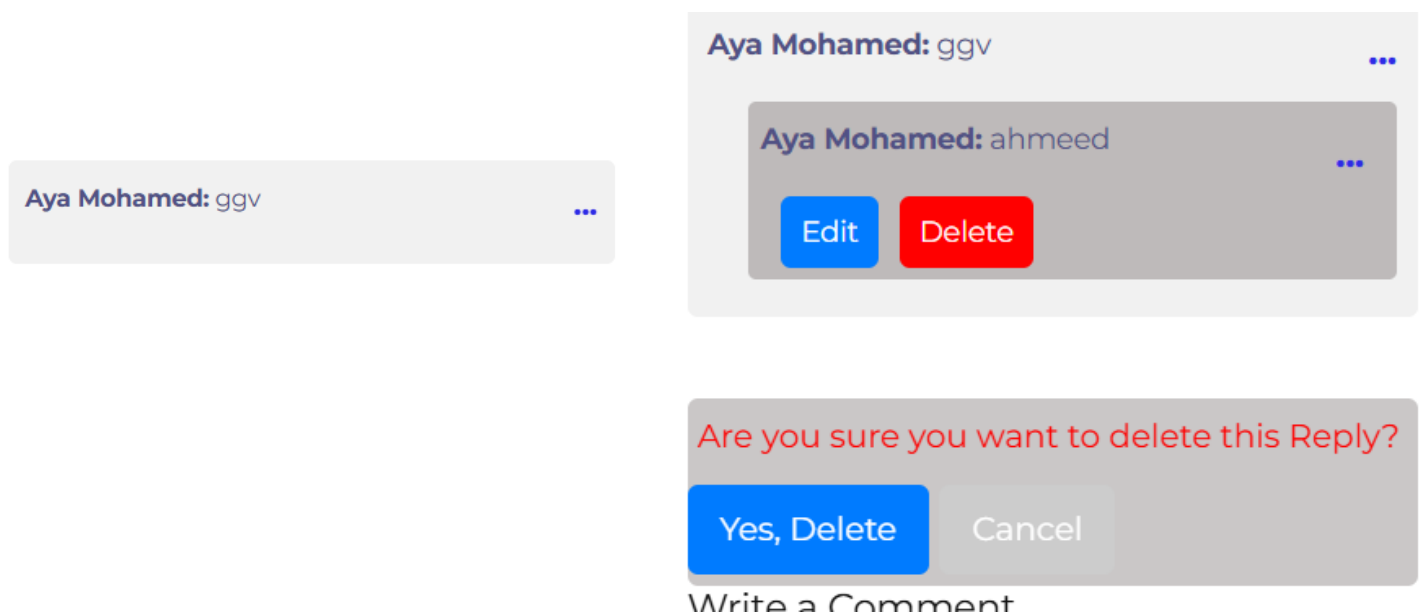
```

The code of Edit Reply

The updateReply function allows a user to update their own reply to a comment. It validates the new content, checks if the reply exists and belongs to the authenticated user, and then updates the reply in the replies table. If successful, it returns a success message.

- **Delete Replies:**

Users can delete their replies to comments, removing their contribution from the thread if they choose to retract it or if they no longer wish to



```

public function deleteReply($commentId, $replyId): JsonResponse|mixed {

    $reply = DB::table('replies')
        ->where(column: 'id', operator: $replyId)
        ->where(column: 'comment_id', operator: $commentId)
        ->where(column: 'user_id', operator: Auth::id())
        ->first();

    if (!$reply) {
        return response()->json(data: [
            'message' => 'Reply not found or you are not authorized to delete this reply',
            'status' => 'error'
        ], status: 404);
    }

    DB::table('replies')->where(column: 'id', operator: $replyId)->delete();

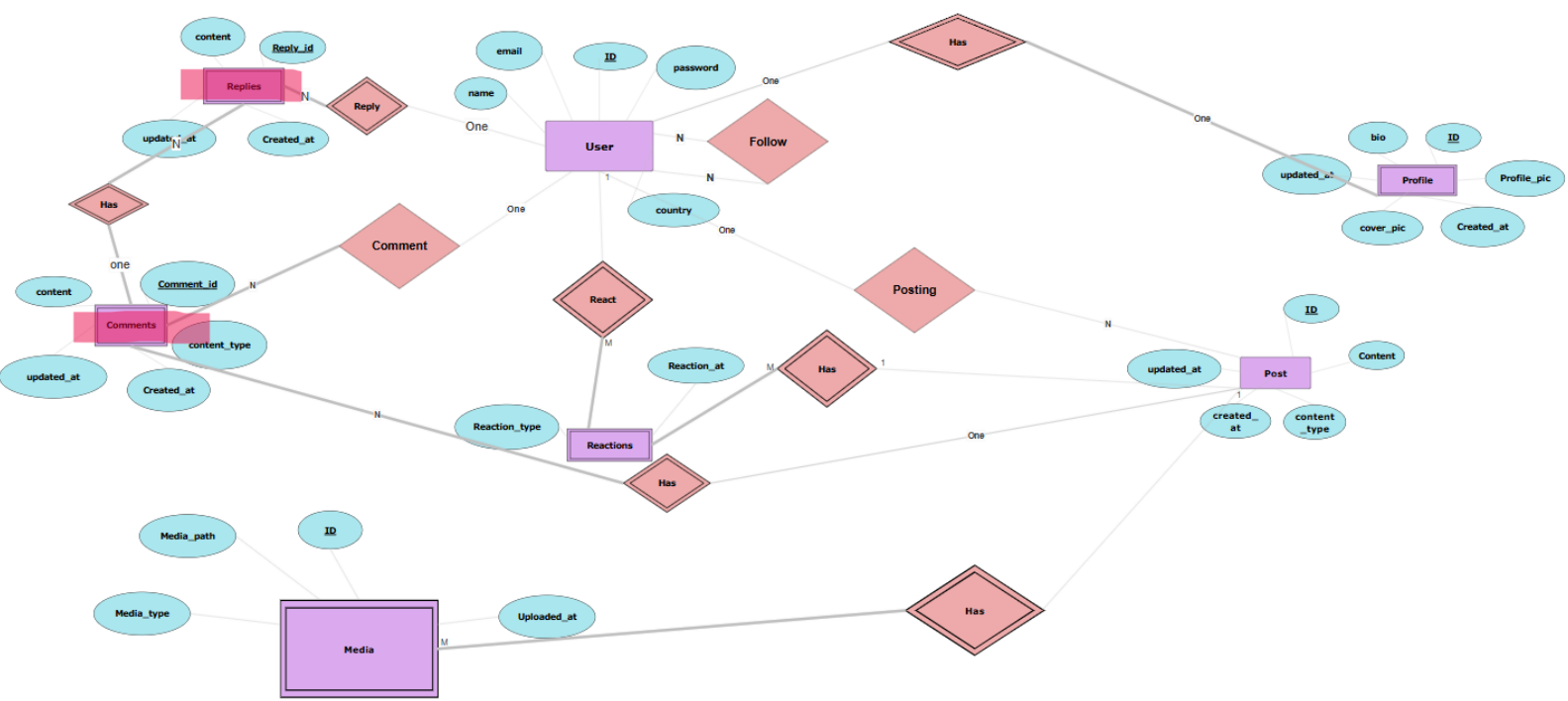
    return response()->json(data: [
        'message' => 'Reply deleted successfully',
        'status' => 'success'
    ], status: 200);
}

```

The code of delete Reply

The deleteReply function allows a user to delete their own reply to a comment. It checks if the reply exists and belongs to the authenticated user, then deletes the reply from the replies table. If successful, it returns a success message; otherwise, it returns an error message.

ER Diagram of Comment and Reply Entity with other entities



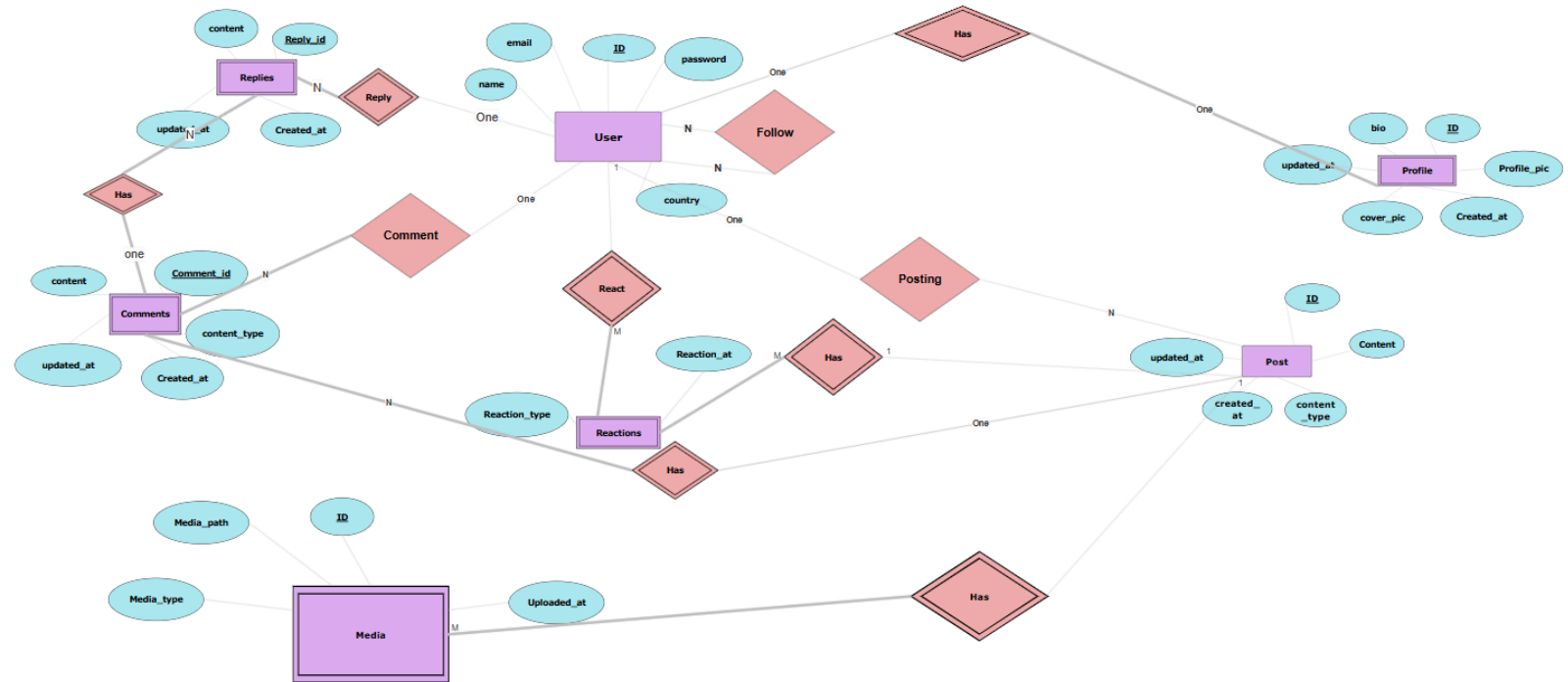
Comment Table From MYSQL

☐		Edit		Copy		Delete	6	ب	text	2	63	2024-12-13 14:36:24	2024-12-13 14:36:24
☐		Edit		Copy		Delete	7	الحمد لله	text	2	63	2024-12-13 14:40:44	2024-12-13 14:40:44
☐		Edit		Copy		Delete	8	اخيرا	text	2	63	2024-12-13 14:42:54	2024-12-13 14:42:54
☐		Edit		Copy		Delete	9	لا	text	2	62	2024-12-13 14:43:09	2024-12-13 14:43:09
☐		Edit		Copy		Delete	10	له	text	2	62	2024-12-13 14:43:14	2024-12-13 14:43:14

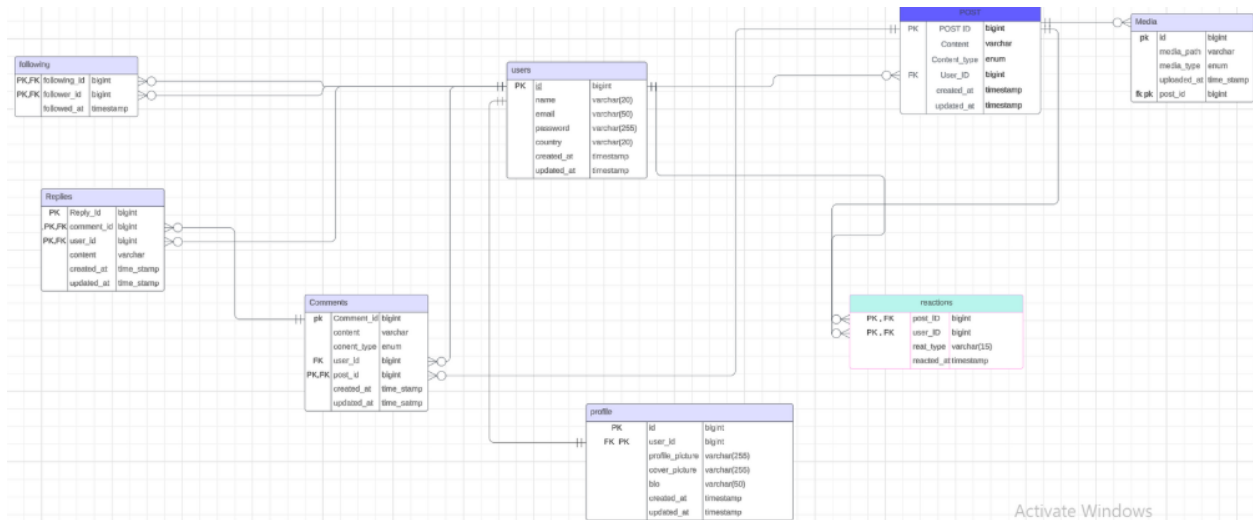
Reply Table From MYSQL

⏪ ⏩			▼ id	comment_id	user_id	content	content_type	created_at	updated_at
☐	✎ Edit 📄 Copy 🗑 Delete		3	70	2	dhj	text	2024-12-13 20:14:07	2024-12-13 20:14:07
☐	✎ Edit 📄 Copy 🗑 Delete		19	77	2	hdeh	text	2024-12-14 21:48:29	2024-12-14 21:48:29

ER Diagram For the Project



UML Diagram For the Project



Schema Diagram For the Project

